CASS Distributed Network Clustering in PySpark

Mackenzie Mallett, Vinay Kumar Devarakonda,
Likkhith Krishnamurthy Aditya, and Arvind Sharma

University of Victoria, Victoria BC, Canada
{mallett.mack, devarakonda207, likkhithkaditya5,
arvind1112sharma}@gmail.com

Abstract. In this paper, we will discuss the application of the CASS (Clustering Algorithm based on Structural Similarity) algorithm used to solve the distributed network clustering problem with the help of PySpark. Communities within the large-scale network are determined by calculating the structural similarity between adjacent vertices, pruning of weak edges according to a certain threshold value, and connected components discovery as communities. As can be seen from the title of the proposed approach, the key to solving the problem consists in transforming it into a database problem with the help of JOIN operations and Bloom filters that allow to efficiently search triangles within graphs. Four datasets are considered in the experiments, which differ in terms of the number of vertices varying from 986 up to 875,713. Emphasis is made on comparing the results received on a single computer and in cluster-based settings.

Keywords: Network Clustering · CASS Algorithm · Structural Similarity · PySpark · Distributed Computing · Bloom Filter · Graph Analytics

1 Introduction

Graph clustering or community detection involves the identification of clusters or communities that are formed by highly interconnected nodes within the graph. It is an important problem in areas like social network analysis, email interaction, biological network modeling, and World Wide Web link analysis. With today's real-world networks growing to consist of millions of nodes and edges, scalability in detection is necessary.

The traditional methods for solving this problem include spectral clustering, modularity maximization, and label propagation. While these methods are effective for smaller graphs, they lack scalability for larger graphs because of memory issues, long computing times, and the inability to split tasks. Even some fast single machine algorithms such as SCAN++ are limited by an absolute limit beyond which there is no scalability.

This issue can be tackled using the CASS algorithm which operates in the Apache Spark environment. The fundamental principle behind CASS is that

counting common neighbors between any two nodes is essentially counting the number of triangles that exist in which the connecting edge exists. Thus, the need for computing neighbor lists is eliminated through the use of CASS which relies on performing two JOINs on the list of edges — effectively transforming the problem of graph computations into a more database-oriented problem solved efficiently by PySpark. The use of a Bloom filter to prune false candidates between the JOINs coupled with shuffle selection helps minimize any issues of data skew arising due to hub nodes.

For this study, we implement CASS on PySpark with four datasets of increasing sizes: Email_EU (986 nodes), Email_Enron (36,692 nodes), Stanford_web (281,903 nodes), and Google_web (875,713 nodes). Comparative results on the performance of CASS are obtained on a single node as well as on a distributed cluster of three nodes. The experimental findings verify that both implementations yield the same results, but the distributed setup provides speedup of up to 46% when used on the biggest dataset.

The rest of this report is organized as follows. Section 2 provides background on graph clustering and distributed computing frameworks. Section 3 describes the three phases of the CASS algorithm. Section 4 details the distributed implementation in PySpark. Section 5 presents the experimental results, and Section 6 concludes the report.

2 Background and Related Concepts

2.1 Foundational Approaches to Clustering and Limitations of Single-CPU Systems

Graph clustering involves partitioning the nodes of a network into clusters wherein intra-cluster connections are more common than inter-cluster connections. This technique has wide applications in social networking analysis, biological networks, monitoring communications, and web links.

There are several traditional algorithms for solving this problem. Spectral clustering relies on eigenvalues of adjacency or Laplacian matrices to determine cluster boundaries. MCL performs simulations using random walks by iteratively expanding and inflating a flow matrix, although MCL must hold the entire adjacency matrix in memory at one time [**?**]. SCAN introduced structural similarity, which is a cosine based measurement of neighborhood overlaps, in classifying nodes into core (having m neighbors above a similarity threshold ϵ), non-core or outliers [2]. SCAN++ [**?**] sped up the process of SCAN by eliminating redundancies while still relying on single-machine calculations.

All these techniques have one major drawback in that they do not extend beyond a single machine. MCL struggles with large graphs, such as AS-SKITTER (1.7M nodes), due to insufficient memory [1]. SCAN++ has no means of distributing tasks across multiple machines. Large graphs, with millions of nodes and edges, require distributed algorithms.

Table 1. Hadoop MapReduce vs Apache Spark for network clustering.

Feature	Hadoop MapReduce	Apache Spark
Intermediate storage	Disk (HDFS)	In-memory (RDD)
Iterative tasks	Slow (disk I/O per step)	Fast (in-memory)
Operations	Map and Reduce only	Join, filter, aggregate, etc.
Broadcast support	Limited	Native broadcast variables

2.2 MapReduce vs Spark for Distributed Computing

The MapReduce computing paradigm involves processing big data sets by a *map* function that generates key-value pairs followed by a *reduce* function that computes aggregate values. The work of PSCAN was in applying SCAN to the MapReduce paradigm, demonstrating that distributed structural clustering was possible. In doing so, PSCAN suffered from the problem that Hadoop MapReduce had to write output to disk between iterations, resulting in a bottleneck. Performance was poor on larger data sets and failed when applied to even larger data sets such as LiveJournal [1].

By keeping data in memory in RDDs, using broadcast variables for broadcast data structures, and performing operations such as joins and filters directly, Apache Spark bypasses writing data to disk between iterations. Table 1 illustrates some of the key differences.

CASS [1] takes advantage of these capabilities by re-casting similarity computations as JOIN operations on edge RDDs and filtering potential candidates with broadcast Bloom filters.

3 CASS Algorithm

The CASS algorithm takes two user-defined parameters: ϵ (epsilon), which serves as the similarity threshold to determine which edges are strong enough to retain, and m, which defines the minimum number of structurally similar neighbours a node must have to be considered a core node. Together, these parameters control the granularity and density of the resulting clusters. A lower epsilon retains more edges and produces larger clusters, while a higher epsilon is stricter and produces smaller, tighter clusters. Similarly, a lower m allows more nodes to qualify as core, increasing cluster coverage, while a higher m requires nodes to be more densely connected before they are considered central to a cluster.

3.1 Phase 1 — Calculating Structural Similarity

The Structural Similarity score given to each edge describes how similar two nodes are in terms of their adjacent nodes. The intuition is straightforward — if two nodes share many of the same neighbours, they are likely to belong to the same community.

$$\text{Structure similarity}(v, w) = \frac{|Nei(v) \cap Nei(w)|}{\sqrt{|Nei(v)||Nei(w)|}} \qquad \text{(Equation 1)}$$

$$V : \text{The vertex set of a graph}$$
$$E : \text{The edge set of a graph}$$
$$\text{Nei}(v) = \{w \in V | (v, w) \in E\} \cup \{v\}$$

A formal definition for similarity score is given in Equation 1. In the numerator we count the number of adjacent nodes that v and w are mutually connected to, or the intersection between the neighbourhoods of both nodes. The denominator normalizes this count by the geometric mean of the neighbourhoods of v and w. The result is a value between 0 and 1, where a score close to 1 indicates that the two nodes share almost all of their neighbours, and a score close to 0 indicates that they share very few.

A key detail in this calculation is that the neighborhood of a node must include the node itself. This ensures that intersection of neighbourhoods reflects the fact that the nodes are connected to one another. This also prevents the denominator from becoming zero when a node has very few connections.

Similarity Score can be naively computed by loading the full neighbour list of every node into memory and comparing lists for each edge. On large networks with millions of nodes this is impractical — the memory required to store and compare these lists exceeds what any single machine can handle. CASS avoids this problem entirely by observing that counting shared neighbours between two nodes is mathematically equivalent to counting the number of triangles that contain the edge connecting them. If a third node c is connected to both v and w, then c is a shared neighbour of v and w, and the three nodes form a triangle. This equivalence allows CASS to reformulate the neighbourhood comparison problem as a triangle finding problem, which can be solved using JOIN operations on the edge list — a form of computation that Apache Spark handles naturally and efficiently at scale. The details of this distributed triangle finding process are described in Section 4.1.

3.2 Phase 2 — Reconstructing the Network

The second phase of CASS uses the similarity scores computed in Phase 1 to reconstruct a cleaner version of the network, retaining only the strongest and most meaningful connections. This phase proceeds in three steps. First, edges with a similarity score below the user-defined threshold ϵ are removed from the network. Removing these nodes rewards density in the network. For example, in Figure 1, nodes 3 and 4 share a similarity score of 0.4, below the epsilon of 0.6 used in this example. This edge is therefore removed, naturally separating the left and right groups of nodes from each other.

Next each remaining node is assigned a label based on the number of connections it has. Nodes with more than m connections are labeled as core, and

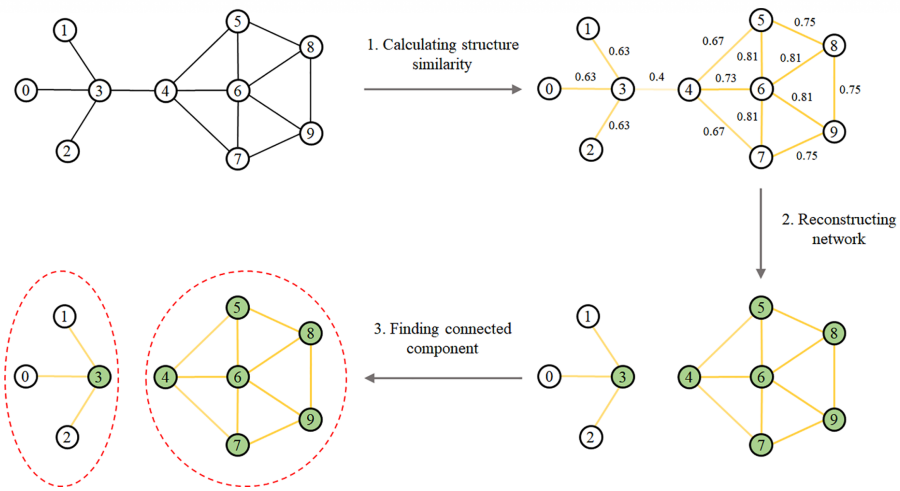

Fig. 1. CASS at a high level

others as non-core. Core nodes are central, well-connected members of a cluster, while non-core nodes sit at the periphery of the network. In Fig 1 core nodes are coloured green and non-core white.

Finally, edges connecting non-core nodes are removed from the network. Imagine two nodes connected only to each other. These nodes do not represent a cluster and should be removed from the network. However, non-core nodes connected to core nodes are simply at the edge of the cluster.

After pruning these nodes the process repeats from labeling nodes as core or non-core. When a pruning iteration removes no more nodes from the network the clusters have converged, and are ready for the final stage of the algorithm.

3.3 Phase 3 — Finding Connected Components

Connected components are groups of nodes where every node can reach every other node through a sequence of edges, with no connections to nodes outside the group. Phases one and two have maximized the density of the network by removing all weak edges and isolated nodes. Each connected component is a cluster and assigned an identifier equal to the minimum node number among all nodes in that component. This convention is particularly well suited to a distributed computing environment. In a distributed system multiple machines may identify connected components simultaneously, and a globally agreed upon naming scheme is needed. Using the minimum node number ensures that all machines independently arrive at the same cluster identifier without requiring any coordination or communication between them.

4 Distributed Implementation Using PySpark

4.1 Triangle Finding via Two JOIN Operations

CASS constructs the triangles need to calculate similarity scores with two inner join operations, supplemented by minor transformation steps. Data begin as an edge list of (source, destination) tuples, referred to as (src, dst) from now on. Figure 2 is sourced directly from the original paper and provides a handy visual map of the calculation. The first join operation creates candidate wedges that

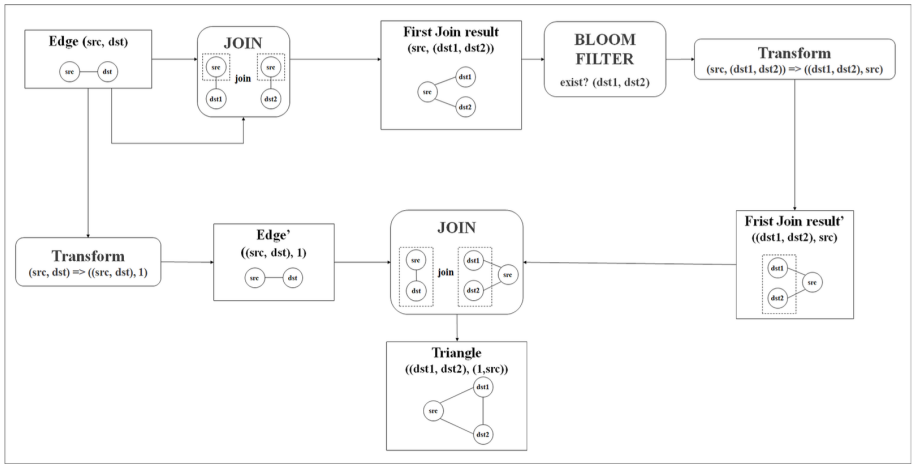

Fig. 2. Structure Similarity Calculation [1]

may eventually become triangles. The edge list is joined to itself so that the tuples (src, dst1) and (src, dst2) combine to become (src, (dst1, dst2)). The second join operation will confirm if dst1 is connected to dst2, thus forming a triangle. In preparation, the output of the first join is transformed from (src, (dst1, dst2)) to ((dst1, dst2), src). Additionally, the (src, dst) tuples in the edge list are transformed to ((src, dst), 1), where the number 1 exists only to match this tuple to the new format of the first join. At this point it may be helpful to re-imagine the ((src, dst), 1) tuple with new labels: ((dst1, dst2), src). The second inner join operation combines the outputs of these two transformations, with a match indicating the existence of a triangle.

These triangles can now be counted. A `.flatMap()` transformation produces (src, dst), 1) tuples each time src and dst are connected in a triangle, and a `.reduceByKey()` action amalgamates these tuples together.

4.2 Shuffle Selection

In a distributed system, each join operation requires a shuffle, where all records sharing a join key are co-located on the same physical machine. High degree hub

nodes create a bottleneck as the RAM tries to process such a large packet of data while smaller packets wait in line. Shuffle selection is implemented immediately before the first join operation to balance the distribution of values associated with each key.

In a network a single hub node could be connected to many outer nodes, creating many values for that key. However, unless each of those nodes is also a hub of a similar degree, then a reverse adjacency list with the outer node as the key would have fewer values. This new construction would have a much more even distribution of values associated with each key.

CASS implements this by counting the number of distinct src and dst values in the edge list of (src, dst) tuples. If there are more distinct values of dst than of src, we can conclude that the key-value pairs involved in joining an inverse of the original edge list to itself would mean joining keys with less associated values than if we had kept the original edge list. If there are more distinct values for src than for dst then the original edge list is already in optimal form for an inner join.

4.3 Bloom Filter Optimization

The output of the first join is a wedge representing a source node and two destination nodes to which it connects. If these destination nodes are connected to each other, then this wedge will form a triangle; but, if not, this wedge is irrelevant to the rest of the calculation. The transformation of the RDD following the first join will require results to be shuffled throughout the network, and the communication cost associated with this should not be spent on wedges which will not form triangles. To filter out unsuccessful candidate wedges CASS introduces a bloom filter immediately after the first join and before the next shuffle.

A bloom filter is a space-efficient, probabilistic structure to distinguish if an element is probably a part of a set or definitely not a part of that set. It consists of a bit array of m bits and k hash functions. When the bloom filter ingests the original edge list, each element of that RDD is fed into each of the k hash functions, which will produce an address in the bit array. That address in the array is set to 1. Once the bit array is constructed it can be used as a filter to test if a candidate (dst1, dst2) edge could belong to the original edge list. The candidate element is fed into the same k has functions, and the resulting addresses are checked against the bit array. If any of the addresses contain a 0, then the candidate element is definitely not a part of the set. If all the bits are one, the candidate element may be a member of the set. This means that a bloom filter has a risk of false positives but no risk of false negatives. The risk of false positives can be set in PySpark's `BloomFilter` class.

The rational for choosing a bloom filter in this implementation is simple. BloomFilters carry no risk of false negatives, ensuring that no valid edges are lost. Although some false positives will exist, the number of false candidate wedges that pass through the filter will always be less than the initial amount,

resulting in a material reduction in the amount of shuffling that must occur in the following transformation.

4.4 Managing RDD's in a Distributed Environment

On a large graph our PySpark implementation creates many data structures including a BloomFilter and many RDD's. As these data structures accumulate they consume the memory executors have available for computation and shuffle orchestration. If a large packet of information arrives at an executor in this state then it will be unable to efficiently receive the packet, causing disk spilling. During phase 2 of the algorithm core and non-core labels are stored in an RDD. In the next iteration some of these nodes will be gone, having been removed during pruning, and the remaining nodes will get new labels. Since the RDD will not be needed in the next iteration, we `.unpersist()` it after it is used for pruning.

A main feature of Spark is the ability to declare an RDD through transformations but save construction for the moment it is needed. This greatly saves on the amount of memory stored across the system. But, it can create a bottleneck if a large, un-constructed RDD is needed for a calculation. When an executor goes to do the calculation it will need to construct the previous RDD as well, potentially exceeding the RAM available for the new calculation. This can be prevented by caching, and then preforming an action on, an RDD before using it later. This is a memory efficient way to make RDD's available for calculations. To mitigate the trade off of computing an action just for the RDD to be available we use a "for each value do nothing" style statement.

```
rdd.cache()
rdd.foreach(lambda x: None)
```

The BloomFilter and degree of each node are data structures that need to be referenced by executors across the distributed system; but do not need to be modified. To minimize the amount of data sent across the network, we use broadcast variables. Broadcast variables send a read-only copy of the structure to the block memory of each executor. This way, when a node needs this information it is immediately available without needing to be sent across the network.

Efficient data structure management is critical to running the CASS algorithm in a distributed environment. In this environment, the true cost is not the calculations themselves, but the communication cost of shuffling packets of information across the network. We efficiently manage the data created during this algorithm to ensure speed and prevent crashing.

4.5 Single node vs Triple node Spark Setup

This report evaluates the performance of the CASS algorithm across two distinct environments: a localized Single-Node setup and a Distributed Triple-Node cluster (comprising one Master and two dedicated Workers). Transitioning from

a single machine to a cluster introduces a fundamental trade-off: while the system gains horizontal scaling through additional CPUs, it also incurs significant network latency and communication overhead.

Each node in this experiment is equipped with 16GB of RAM and 8 logical cores (4 vCPUs). In the single-node environment, the Spark Driver and Executor processes coexist on the same physical hardware, competing for a shared 16GB pool. This necessitates a conservative memory profile to prevent the Operating System from killing processes due to memory exhaustion.

In contrast, the triple-node cluster allows for Process Isolation. Since the Driver resides on the Master and Executors reside on the Workers, each can be allocated a larger portion of their respective node's 16GB without contention. This is reflected in the increased `spark.executor.memory` (from 6GB to 10GB) and `spark.driver.maxResultSize` (from 2GB to 4GB) in the cluster configuration.

In distributed mode, the Master node acts as the orchestrator. If worker nodes produce large intermediate result sets that exceed the Driver's ability to aggregate or serialize, the system may experience Heartbeat Timeouts. YARN's resource manager often interprets these hangs as a node failure, preemptively shutting down the application. To mitigate this, we implemented two specific optimizations:

- **Kryo Serialization:** Used to reduce the size of the objects being sent across the network.
- **Network Timeout Extensions:** Increased `spark.network.timeout` to 800 seconds to accommodate heavy shuffle phases.

The number of partitions used in this report differ from the original CASS paper. This report sets 24 partitions for the single machine experiment and 48 for the distributed system experiment. Custom parameters allow each system to perform at its best, and leads to a fair comparison.

5 Results

5.1 Dataset and User Defined Variables

This report evaluates the speed of the CASS algorithm on four datasets of increasing size, as shown in Table 2. ϵ was set to 0.7 and m to 2. The Bloom Filter was given a false positive rate of 0.3. These settings were chosen to match the environment variables described in the original CASS paper[1].

5.2 Evaluating CASS Performance

Cluster quality is evaluated on the remaining node ratio and the average normalized cut for each dataset. Not only do these metrics indicate cluster health, they can be used to verify that both the distributed and single machine environments produce similar clusters.

Table 2. Dataset characteristics.

Dataset	Nodes	Edges	Description
Email_EU	986	16,064	European email network
Email_Enron	36,692	183,831	Enron email corpus
Stanford_web	281,903	1,992,636	Stanford web graph
Google_web	875,713	4,322,051	Google web graph

Table 3. Remaining Node Ratio

Dataset	Cluster (s)	Single Machine (s)
Email_EU	0.9320	0.9320
Email_Encore	0.7210	0.7210
Stanford_web	0.5004	0.5004
Google_web	0.6500	0.6500

The remaining node ratio is the proportion of nodes successfully assigned to a community relative to the total node count. A higher ratio indicates broader coverage, while a lower ratio suggests more stringent membership criteria where only the most densely connected cores are retained. As shown in Table 3, the implementation maintains high coverage for dense datasets like Email_EU.

Average normalized cut evaluates the separation between communities by comparing the external connectivity of a cluster to its internal density. Dividing the number of edges connecting a cluster to the rest of the network by the total degree of the nodes within that cluster provides a normalized measure of this separation. Low values indicate "tight" clusters with few edges escaping to other communities. As shown in Table 4, this implementation produces values comparable to the original CASS paper [1].

Furthermore, both experiments produced exactly the same values for each metric. This indicates that the distributed environment can reliably produce the same results as a single machine and that the random shuffling of Spark does not affect the deterministic nature of the CASS algorithm.

5.3 Cluster vs Single Machine Performance

The benefits of operating in a distributed environment are best realized on massive datasets. On smaller datasets the communication cost of shuffling information often matches or exceeds the benefits of parallel computation.

Our results in Table 5 are consistent with this expectation. As the size of the data set increases the time saved as a proportion of the time on a single machine increases in kind. This difference is most noticeable in the jump from 183,831 edges in the Email_Enron data to the 1,992,635 edges in the Stanford_web data set. This order of magnitude may represent the tipping point between communication cost and the realized benefits of parallel computing.

Table 4. Average Normalized Cut

Dataset	Cluster (s)	Single Machine (s)
Email_EU	0.0628	0.0628
Email_Encore	0.1783	0.1783
Stanford_web	0.2950	0.2950
Google_web	0.3473	0.3473

Table 5. Execution time comparison between cluster and single machine.

Dataset	Cluster (s)	Single Machine (s)	Speedup (%)
Email_EU	111.6	114.6	2.62
Email_Encore	158.6	172.9	8.27
Stanford_web	3084.0	5120.3	39.77
Google_web	833.2	1551.5	46.30

6 Conclusion

This report successfully implemented and evaluated the CASS algorithm within a distributed PySpark environment, demonstrating its efficacy for large-scale network clustering. By reformulating structural similarity as a series of database JOIN operations, the implementation effectively bypassed the memory constraints of single-machine systems. Key optimizations—including Bloom filters to prune unsuccessful candidate wedges, Shuffle Selection to mitigate hub-node bottlenecks, and the strategic use of broadcast variables—were instrumental in managing the communication overhead inherent in distributed systems.

Experimental results across four datasets confirmed that the distributed approach maintains perfect mathematical parity with single-machine results while offering significant performance gains as network scale increases. While smaller datasets showed negligible improvements due to Spark's scheduling and networking overhead, larger graphs like the Google web graph achieved a speedup of 46.30%. Ultimately, this work validates CASS as a robust, deterministic, and scalable solution for community detection in massive, real-world networks.

Code for this project is available at:
https://github.com/mack-mallett/Implementing-CASS-in-PySpark

References

1. Kim J, Shin M, Kim J, Park C, Lee S, Woo J, et al. (2018) CASS: A distributed network clustering algorithm based on structure similarity for large-scale network. PLoS ONE 13(10): e0203670. https://doi.org/10.1371/journal. pone.0203670
2. Shiokawa H, Fujiwara Y, Onizuka M. SCAN++: efficient algorithm for finding clusters, hubs and outliers on large-scale graphs. Proceedings of the VLDB Endowment, 2015; 8(11): 1178–1189.
3. Apache Spark Documentation, `https://spark.apache.org/docs/latest/`

4. Ngu, H.C.V., Huh, J.-H.: B+-tree construction on massive data with Hadoop. Cluster Computing **22**(S1), 1011–1021 (2017)